Vade Mecum User Guide

1. Getting started 2
1.1. Setting up 2
1.2. Features 2

2. Available settings 3
2.1 `lang` 3
2.2 `font` 3
2.3 `font-size` 3
2.4 `heading-numbering` 3
2.5 `page-numbering` 3
2.6 `page-number-alignment` 3
2.7 `bullet-point-indent` 3
2.8 `use-hydra` 4
2.9 `hydra-margins` 4
2.10 `justify` 4
2.11 `code-font` 4
2.12 `code-font-size` 4
2.13 `code-bg-color` 4
2.14 `codly-config` 5
2.15 `syntax-theme` 5
2.16 `math-font` 5
2.17 `math-counter-levels` 5
2.18 `math-numbering` 5

3. Utilities 6
3.1. Functions 6
3.1.1. advanced-outline 6
3.1.2. code-block 7
3.1.3. float-table-from-2d-array 7
3.1.4. int_round 8
3.1.5. is-content-empty 9
3.1.6. is-content-non-empty 9
3.1.7. table_from_columns 9
3.2. Custom frame utilities 11
3.2.1. hidden 11
3.2.2. hidden-no-title 11
3.2.3. hidden-sup-only 11
3.2.4. pre-post-style 12
3.2.5. setup-frame-environment 12
3.3. Math symbols 13

4. Environments 15
4.1. Math Environments 15
4.5. Custom Environments 18

1. Getting started

1.1. Setting up

To get started, you can use the following snippet in your document:

```typ
#import "@local/vade-mecum:0.1.0": *

#show: e.set_(
    settings,
    // Configuration options are set here
    font: "Fira Sans",
    heading-numbering: "1.",
    code-font-size: 12pt,
    justify: false,
    // etc.
)

#show: template
```

1.2. Features

This package is a personal all-in-one Typst template, with the following features:

- Configurable basic settings. See Section 2 for a list of available settings.
- Opinionated Hydra integration for section headings in page headers.
- Styling for certain Typst elements, including links, outlines, math symbols and references.
- Includes Itemize's list/enum fixes and referencing (which is still not possible in Typst)
- Includes big-todo for **TODO: notes** in the document.
- Uses Codly with opinionated defaults for code blocks.
- A bunch of utility functions. See Section 3 for a list of available functions.
- A bunch of shorthand math symbols. See Section 3.3 for a list of available symbols.
- Environments for math use cases, using frame-it See Section 4.1 for more information.
- Utilities for building custom frame-it frames. See Section 4.5 for an example, and see Section 3.2 for the reference.
- A function `advanced-outline` for building more advanced outlines that are better integrated with the other features of this package.

2. Available settings

`lang` 2.1

Type: string
Overall language of the document. This is used by Typst to determine hyphenation and other language-specific settings.
Default value: en

`font` 2.2

Type: string, array or dictionary
Main font of the document. Does not apply to Math mode nor Raw mode.
Default value: `("Linux Libertine O", "Libertinus Serif")`

`font-size` 2.3

Type: length
Font size for the main document.
Default value: `12pt`

`heading-numbering` 2.4

Type: function or string
Heading numbering pattern. Uses same pattern as Typst's numbering function.
Default value: 1.1.

`page-numbering` 2.5

Type: function or string
Page numbering pattern. Uses same pattern as Typst's numbering function.
Default value: 1

`page-number-alignment` 2.6

Type: alignment
Alignment of page numbers in the page footer.
Default value: `center`

`bullet-point-indent` 2.7

Type: length
Default indentation for Typst's enumerations and lists.
Default value: `1em`

`use-hydra` 2.8

Type: boolean
Whether to use Hydra for section headings in page headers. By default, the first level heading is shown at the left of the page header, and the second level heading is shown at the right of the page header. Note that this option will force each section to start on a new page, and that Hydra won't show in the first page of each section.
Default value: `false`

`hydra-margins` 2.9

Type: alignment or dictionary
Adapted page margins used when Hydra is enabled. This is mostly useful for increasing the top margin to make more room for the header.
Default value: `(top: 8em)`

`justify` 2.10

Type: boolean
Whether to justify text or not.
Default value: `true`

`code-font` 2.11

Type: string, array or dictionary
Font used for code blocks.
Default value: `("FiraCode Nerd Font", "DejaVu Sans Mono")`

`code-font-size` 2.12

Type: length
Font size used for code blocks.
Default value: `9pt`

`code-bg-color` 2.13

Type: color
Background color for code blocks.
Default value: `rgb("#f6f8fa")`

`codly-config` 2.14

Type: arguments
Default configuration arguments for Codly code blocks.
Default value: `arguments(zebra-fill: none)`

`syntax-theme` 2.15

Type: path
Syntax theme for code blocks. This should be a path to a .tmtheme file.
Default value: `path("/src/themes/Github Light.tmtheme")`

`math-font` 2.16

Type: string, array or dictionary
Font used for mathematical content.
Default value: `("Erewhon Math", "New Computer Modern Math")`

`math-counter-levels` 2.17

Type: integer
Levels of numbering dependence for math frames. This determines how many levels of numbering depend on the section hierarchy. For example, if this is set to 2, the first two counters in the numbering pattern of math frames will use the section counters, while the rest will increment independently. This last counter will be reset at the beginning of each section.
Default value: 0

`math-numbering` 2.18

Type: none or string
Numbering pattern for math frames. Uses the same pattern as Typst's numbering function.
Default value: 1.1

3. Utilities

This section acts as a reference for the utility functions provided by this package.

3.1. Functions

3.1.1. advanced-outline

Creates an outline of the document with advanced options for formatting and filtering.

Parameters

```
advanced-outline(
  indent_figures: bool auto,
  hide_figure_sup: bool auto,
  max_bold_headings_level: int auto,
  fill: content,
  targets: array<element>,
  ..kwargs
) -> content
```

indent_figures `bool` or `auto`

Whether to automatically indent figures in the outline. Stock Typst always considers figures as top-level elements, which is undesirable if the outline combines headings and figures. If set to `auto`, figures will be indented if headings are included in the outline.

Default: `auto`

hide_figure_sup `bool` or `auto`

Whether to hide the supplement for figures in the outline. This is useful if the outline is only for one kind of figure, as the supplement will be the same for all of them. If set to `auto`, the supplement will be hidden if the outline is only for one kind of figure.

Default: `auto`

max_bold_headings_level `int` or `auto`

The maximum level of headings to be displayed in bold in the outline. If set to `auto`, only the first level of headings will be bolded if the outline is only for headings. Otherwise, if other elements are included in the outline, all headings will be bolded.

Default: `auto`

fill `content`

The fill style for the outline entries.

Default: `line(length: 100%, stroke: .2pt + gray)`

targets `array<element>`

The target elements to include in the outline.

Default: `(heading,)`

..kwargs

Additional keyword arguments to pass to Typst's built-in `outline` function.

3.1.2. code-block

Returns a code block with the contents of a file, using the filename as the header and its extension as the language for syntax highlighting. The code block is breakable, allowing it to span multiple pages if necessary.

NOTE: The language detection based on the file extension currently does NOT work. Prior to Typst 0.15, paths did not exist and thus the user had to pass the raw contents of the file to this function, along with the file name. Now, path support exists, but no operations are allowed with them, including extracting their string representation. For now, the extension will have to be manually specified

Parameters

```
code-block(
  filename: path,
  lang: str
) -> content
```

filename `path`

The path to the file to read.

lang `str`

Extension of the file to use for syntax highlighting. If empty, no syntax highlighting will be applied.

Default: `""`

3.1.3. float-table-from-2d-array

Creates a table from a 2D array of data, with optional headers for the rows and columns.

Parameters

```
float-table-from-2d-array(
  data: array<content>,
  header_up: array<content string>,
  header_side: array<content string>,
  kwargs: arguments none,
  precision: int
) -> content
```

data `array<content>`

The 2D array of data to populate the table.

header_up `array<content` or `string>`

The header for the columns (top row). If none, the table will have no header row.

Default: `none`

header_side `array<content` or `string>`

The header for the rows (first column). If none, the table will have no header column.

Default: `none`

kwargs `arguments` or `none`

Additional keyword arguments to pass to the table function. If `none`, defaults to center alignment and equal column widths.

Default: `none`

precision `int`

The number of decimal places to round the data to.

Default: `3`

3.1.4. int_round

Rounds a number to a specified precision and returns it as a string.

Parameters

```
int_round(
  value: number,
  precision: int
) -> str
```

value `number`

The number to round.

precision `int`

The number of decimal places to round to.

Default: `3`

3.1.5. is-content-empty

Checks if a Content is empty (i.e., has no text)

Parameters

```
is-content-empty(h: content) -> bool
```

h `content`

The content to check.

3.1.6. is-content-non-empty

Checks if a content is non-empty (i.e., has text)

Parameters

```
is-content-non-empty(h: content str) -> bool
```

h `content` or `str`

The content to check.

3.1.7. table_from_columns

Creates a table from a list of columns, as opposed to a list of rows.

Parameters

```
table_from_columns(
  kwargs: arguments | none,
  ..columns: content
) -> content
```

kwargs `arguments` or `none`

Additional keyword arguments to pass to the table function.

..columns `content`

The columns to use for the table. Each column should be a list of cells.

3.2. Custom frame utilities

3.2.1. hidden

No-frame style that shows all attributes

Parameters

```
hidden(
  title,
  tags,
  body,
  supplement,
  number,
  arg
)
```

3.2.2. hidden-no-title

No-frame style that omits the title.

Parameters

```
hidden-no-title(
  title,
  tags,
  body,
  supplement,
  number,
  arg
)
```

3.2.3. hidden-sup-only

No-frame style that only shows the supplement

Parameters

```
hidden-sup-only(
  title,
  tags,
  body,
  supplement,
  number,
  arg
)
```

3.2.4. pre-post-style

Frame style that adds pre and post content around the given style.

Parameters

```
pre-post-style(
  pre_content: content function none,
  style: function,
  post_content: content function none
)
```

pre_content `content` or `function` or `none`

The content to add before the frame. It can be either a fixed content or a function with the same signature as the style function: `(title, tags, body, supplement, number, arg) ⇒ content`

style `function`

The frame style to wrap. It should have the signature `(title, tags, body, supplement, number, arg) ⇒ content`

post_content `content` or `function` or `none`

The content to add after the frame. It can be either a fixed content or a function with the same signature as the style function: `(title, tags, body, supplement, number, arg) ⇒ content`

3.2.5. setup-frame-environment

Setup environment for figures using frame-it. Since this applies show rules, it requires to be called like this: `#show: doc ⇒ setup-frame-environment(doc, ...)`

Parameters

```
setup-frame-environment(
  doc: content,
  kind: str,
  style: function,
  numbering: function none,
  levels: int,
  breakable: bool,
  alignment: enum
) -> none
```

doc `content`

The document content to configure.

kind `str`

The kind of figure to configure.

style `function`

The frame style to apply. It should have the signature `(title, tags, body, supplement, number, arg) → content`.

numbering `function` or `none`

The numbering scheme to use for the figure. If `none`, no numbering is applied.

Default: `"1.1"`

levels `int`

The number of levels to use for dependent numbering. If `numbering` is `none`, this is ignored.

Default: `0`

breakable `bool`

Whether the figure should be breakable across pages.

Default: `true`

alignment `enum`

The alignment of the figure content. Options are `left`, `center`, `right`.

Default: `left`

3.3. Math symbols

The following shorthands are provided for common math environments:

- `AA, BB, ..., ZZ` for the corresponding blackboard bold letters.
- `scr` for script (roundhand) font style. I need this one because my default font has no support for it, so I just fallback to another font.
- `suchthat` for a custom "such that" slash symbol.

- `sfrac, hfrac` for skewed and horizontal fractions, respectively.
- Other symbols have also been tweaked.

4. Environments

4.1. Math Environments

The following shows an example usage of the math environments provided by this package.

```typ
#import "@local/vade-mecum:0.1.0": *

#show: e.set_(
  settings,
  heading-numbering: "A.1.",
  math-numbering: "A.1",
  math-counter-levels: 2,
  font-size: 10pt,
)
#show: template

== Definitions and notation

#notation[
    We write $NN = {0, 1, 2, dots}$ for the natural numbers, $ZZ$ for the
    integers, $QQ$ for the rationals, $RR$ for the reals and $CC$ for the
    complex numbers.
]

#def[Metric space][
    A *metric space* is a pair $(X, d)$ where $X$ is a set and $d: X times X
    -> RR$ satisfies, for all $x, y, z in X$:
    + $d(x, y) >= 0$, with equality iff $x = y$,
    + $d(x, y) = d(y, x)$,
    + $d(x, z) <= d(x, y) + d(y, z)$.
] <def:metric>

#def[Open ball][
    For $x in X$ and $r > 0$, the *open ball* is $ B_r (x) = {y in X
    suchthat d(x, y) < r}. $
]

== Results

#lemma[Balls are open][
    Every open ball $B_r (x)$ in a metric space $(X, d)$ is open.
  #proof[][
      Let $y in B_r (x)$ and set $epsilon = r - d(x, y) > 0$. For $z in
      B_epsilon (y)$, the triangle inequality gives
      $ d(x, z) <= d(x, y) + d(y, z) < d(x, y) + epsilon = r, $
      so $B_epsilon (y) subset B_r (x)$.
```

```
  ]
] <lem:ball-open>


#theorem[Cauchy--Schwarz][
    For all $u, v$ in an inner product space,
    $ |chevron.l u, v chevron.r|^2 ⩽ chevron.l u, u chevron.r chevron.l v,
    v chevron.r. $
  #proof[][
      If $v = 0$ the claim is trivial. Otherwise, for any scalar $t$,
      $ 0 ⩽ norm(u - t v)^2 = norm(u)^2 - 2 t chevron.l u, v chevron.r +
      t^2 norm(v)^2, $
      and choosing $t = sfrac((chevron.l u, v chevron.r) / norm(v)^2)$
      yields the result.
  ]
] <thm:cs>


== Examples and remarks

#example[Discrete metric][
    On any set $X$, define $d(x, y) = 0$ if $x = y$ and $d(x, y) = 1$
    otherwise. Then $(X, d)$ is a metric space in which $B_1 (x) = {x}$ for
    every $x$.
]

#remark[
    By @lem:ball-open, the open balls form a basis for a topology on $X$.
    Compare this with @def:metric and @thm:cs.
]

#remark[Calligraphic and blackboard letters][
    Script letters: $scr(F), scr(G), scr(P)(X)$. Blackboard letters: $AA,
    BB, PP, EE, RR^n$.
    Fractions: $sfrac(1 / 2)$ (skewed) and $hfrac(1 / 2)$ (horizontal).
]
```

D.2. Definitions and notation

Notation: We write $\mathbb{N} = \{0, 1, 2, \dots\}$ for the natural numbers, $\mathbb{Z}$ for the integers, $\mathbb{Q}$ for the rationals, $\mathbb{R}$ for the reals and $\mathbb{C}$ for the complex numbers.

Metric space **Definition D.2.1**

A **metric space** is a pair (X, d) where X is a set and $d : X \times X \to \mathbb{R}$ satisfies, for all $x, y, z \in X$:

1. $d(x, y) \geq 0$, with equality iff $x = y$,
2. $d(x, y) = d(y, x)$,
3. $d(x, z) \leq d(x, y) + d(y, z)$.

Open ball **Definition D.2.2**

For $x \in X$ and $r > 0$, the **open ball** is

$$B_r(x) = \left\{ y \in X \;\middle/\; d(x, y) < r \right\}.$$

D.3. Results

Balls are open **Lemma D.3.1**

Every open ball $B_r(x)$ in a metric space (X, d) is open.

Proof

Let $y \in B_r(x)$ and set $\varepsilon = r - d(x, y) > 0$. For $z \in B_\varepsilon(y)$, the triangle inequality gives

$$d(x, z) \leq d(x, y) + d(y, z) < d(x, y) + \varepsilon = r,$$

so $B_\varepsilon(y) \subseteq B_r(x)$. □

Cauchy–Schwarz **Theorem D.3.1**

For all u, v in an inner product space,

$$|\langle u, v \rangle|^2 \leq \langle u, u \rangle \langle v, v \rangle.$$

Proof

If $v = 0$ the claim is trivial. Otherwise, for any scalar t,

$$0 \leq \|u - t v\|^2 = \|u\|^2 - 2t\langle u, v \rangle + t^2 \|v\|^2,$$

and choosing $t = {}^{\langle u, v \rangle}\!/_{\|v\|^2}$ yields the result. □

D.4. Examples and remarks

Discrete metric **Example D.4.1**

On any set X, define $d(x, y) = 0$ if $x = y$ and $d(x, y) = 1$ otherwise. Then (X, d) is a metric space in which $B_1(x) = \{x\}$ for every x.

Remark D.4.1: By Lemma D.3.1, the open balls form a basis for a topology on X. Compare this with Definition D.2.1 and Theorem D.3.1.

Remark D.4.2: Script letters: $\mathcal{F}, \mathcal{G}, \mathcal{P}(X)$. Blackboard letters: $\mathbb{A}, \mathbb{B}, \mathbb{P}, \mathbb{E}, \mathbb{R}^n$. Fractions: ${}^1\!/_2$ (skewed) and $1/2$ (horizontal).

4.5. Custom Environments

Custom environments can be defined too.

```typ
#import "@local/vade-mecum:0.1.0": *
// ...
#let exercise = frame("Exercise", red, kind: "exercise")
#show: doc => setup-frame-environment(doc,
    "exercise", styles.thmbox,
    numbering: "1", levels: 0, breakable: true,
)

#let solution = frame("Solution", green, kind: "solution")
#show: doc => setup-frame-environment(doc,
    "solution", hidden-sup-only,
    numbering: none, levels: 0, breakable: true,
)

#exercise[][
    Given a list of numbers, write a function that returns the sum of the squares of the even numbers in the list.

    #solution[][
        ```py
        def sum_of_even_squares(numbers):
            return sum(number ** 2 for number in numbers if number % 2 == 0)
        ```
    ]
]
```

Output:

Exercise 1

Given a list of numbers, write a function that returns the sum of the squares of the even numbers in the list.

Solution:

```py
def sum_of_even_squares(numbers):
    return sum(number ** 2 for number in numbers if number % 2 == 0)
```